

Design Patterns and Principles

Part 1 — SOLID Principles

SOLID is a set of 5 rules that make OOP code easier to maintain and extend. Each letter stands for one rule.

S — Single Responsibility Principle (SRP)

One class = one job. If a class is doing too many things, split it up.

- Bad: *Invoice* class that prints, saves to DB, and calculates total
- Good: *Invoice* for data, *InvoicePrinter* for printing, *InvoiceDB* for saving

Easy way to check: can you describe the class in one sentence without using 'and'?

O — Open/Closed Principle (OCP)

Classes should be open for extension but closed for modification. Add new features by writing new code, not by editing old code.

- Add a new payment method by creating a new class, not editing the existing `PaymentProcessor`

```
class PaymentProcessor:
    def process(self, payment: Payment): ...

class UPIPayment(Payment): ... # just add this, don't touch above
```

L — Liskov Substitution Principle (LSP)

If class B extends class A, you should be able to swap A with B and nothing should break.

- Classic gotcha: making `Penguin` extend `Bird` when `Bird` has a `fly()` method — penguins can't fly!
- Fix: have a `FlyingBird` subclass, `Penguin` just extends `Bird` directly

■ *Basically: don't lie about what a subclass can do.*

I — Interface Segregation Principle (ISP)

Don't force a class to implement methods it doesn't need. Keep interfaces small and specific.

- Bad: one big `IMachine` interface with `print()`, `scan()`, `fax()`
- Good: separate `IPrinter`, `IScanner`, `IFax` — a simple printer only implements `IPrinter`

D — Dependency Inversion Principle (DIP)

High-level code shouldn't depend on low-level code directly. Both should depend on an interface.

- Instead of: `NotificationService` directly using `EmailSender`
- Do: `NotificationService` depends on `IMessageSender`, `EmailSender` implements it

This makes swapping implementations (email → SMS → push) super easy.

Part 2 — Creational Patterns

These patterns deal with how objects get created. The goal is to make creation flexible and hide the messy details from the rest of the code.

Singleton

Only one instance of a class exists in the whole app. Everyone shares that one instance.

- Use for: DB connections, config loader, logger

```
class Logger:
    _instance = None

    @classmethod
    def get(cls):
        if not cls._instance:
            cls._instance = cls()
        return cls._instance
```

Watch out — singletons can make unit testing tricky. Use sparingly.

Factory Method

Instead of calling a constructor directly, you call a factory method that decides which class to create.

- Use for: when the exact type of object depends on some condition at runtime
- Example: a DocumentFactory that returns PdfDoc or WordDoc based on file type

Builder

Build complex objects step by step. Good when an object has tons of optional fields.

- Use for: SQL query builder, HTTP request builder, complex form objects

```
user = UserBuilder()
    .set_name('Ravi')
    .set_age(25)
    .set_email('ravi@mail.com')
    .build()
```

Way cleaner than a constructor with 10 parameters!

Part 3 — Structural Patterns

These are about how classes and objects are composed together into bigger structures.

Adapter

Makes two incompatible interfaces work together. Like a power plug adapter.

- Use for: plugging in a 3rd-party library that has a different interface than what your code expects
- Wrap the old class in a new Adapter class that has the interface you need

Decorator

Add new behaviour to an object at runtime without changing its class. Stack decorators like wrappers.

- Use for: adding logging, caching, auth checks around existing functions
- Example: wrap a basic Coffee with MilkDecorator, then SugarDecorator

Python's @decorator syntax is literally this pattern!

Proxy

A stand-in object that controls access to the real object.

- Use for: lazy loading (don't load until needed), access control, caching responses
- Example: ImageProxy that only loads the actual image when it's first displayed

Part 4 — Behavioral Patterns

These deal with how objects communicate and share responsibilities.

Observer

One object (subject) notifies a bunch of other objects (observers) when its state changes.

- Use for: event systems, MVC (view watches model), notification feeds
- Example: YouTube channel (subject) notifies all subscribers (observers) on new video

```
subject.subscribe(observer1)
subject.subscribe(observer2)
subject.notify() # both get called
```

Strategy

Define a bunch of algorithms and make them interchangeable at runtime.

- Use for: sorting (bubble/quick/merge), payment methods, compression types
- Example: set `sorter.strategy = QuickSort()` or `BubbleSort()` and call `sorter.sort()`

This is basically the OCP in action — add a new strategy without touching old code.

Command

Wrap a request as an object. This lets you queue, log, or undo requests.

- Use for: undo/redo in editors, task queues, GUI button actions
- Each action (copy, paste, delete) becomes a Command object with `execute()` and `undo()`

Part 5 — Architectural Patterns

Bigger-picture patterns for how to structure a whole application.

MVC — Model View Controller

- Model: data and business logic
- View: what the user sees (UI)
- Controller: handles user input, updates model, picks which view to show

Used in basically every web framework — Django, Spring, Rails, etc.

Main benefit: you can change the UI without touching the business logic.

Dependency Injection (DI)

Instead of a class creating its own dependencies, they get passed in from outside.

- Bad: class UserService that creates its own DBConnection inside
- Good: DBConnection is passed into UserService via constructor

```
# Bad
class UserService:
    def __init__(self):
        self.db = DBConnection() # tightly coupled!

# Good
class UserService:
    def __init__(self, db): # injected from outside
        self.db = db
```

Makes unit testing easy — just inject a mock/fake dependency.

Quick Reference Cheat Sheet

When to use what?

- **Singleton** — need one shared instance globally
- **Factory** — object type decided at runtime
- **Builder** — object with many optional params
- **Adapter** — making incompatible interfaces fit
- **Decorator** — adding features without changing class
- **Proxy** — controlling access or lazy loading
- **Observer** — one-to-many event notification
- **Strategy** — switching algorithms at runtime
- **Command** — undo/redo or queuing actions

Useful links

- SOLID deep dive: <https://www.baeldung.com/solid-principles>
- All patterns with diagrams: <https://refactoring.guru/design-patterns>
- Medium overview: <https://medium.com/@softwaretechsolution/design-pattern-81ef65829de2>

— end of notes —